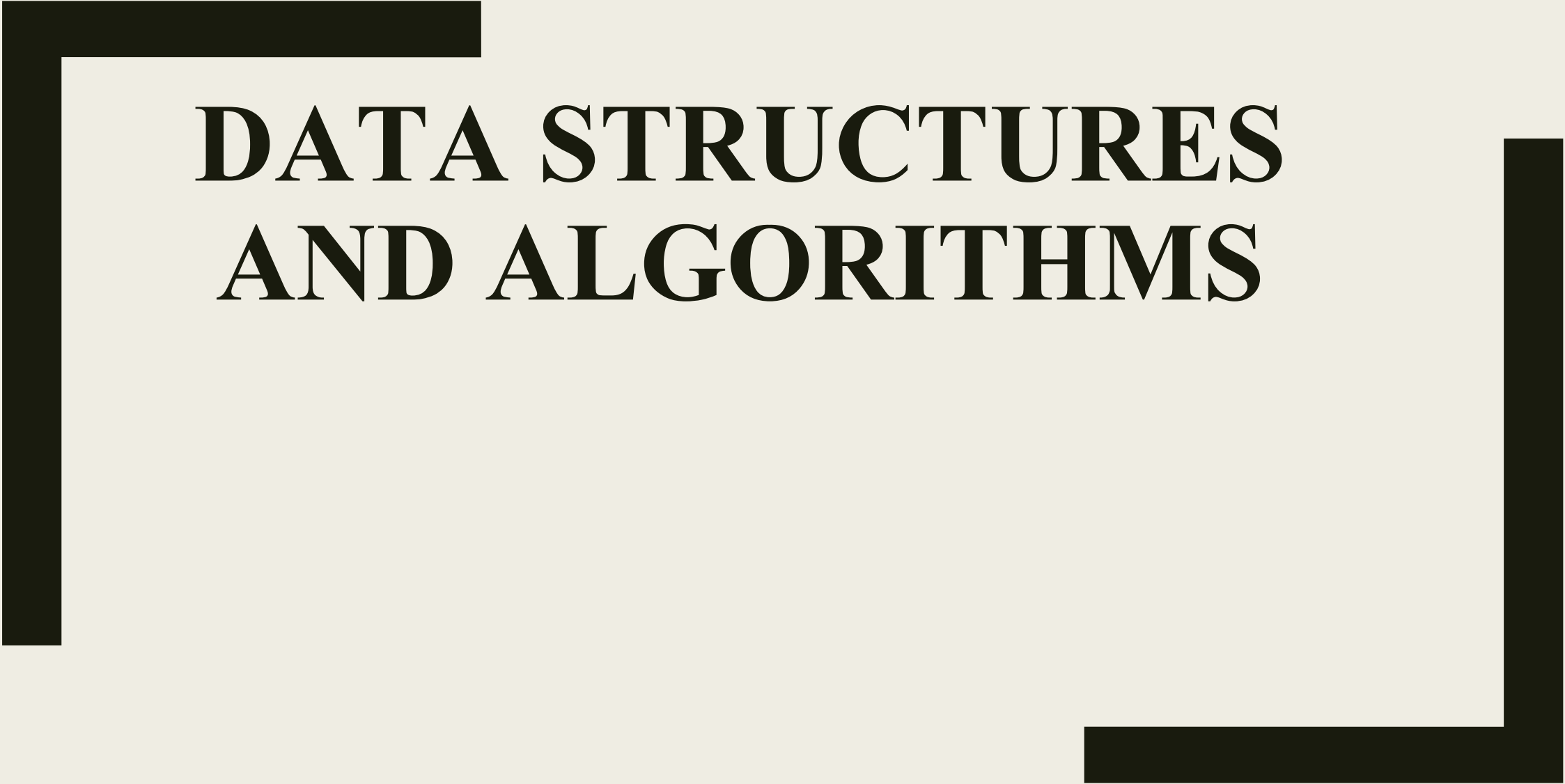


A thick black L-shaped frame is positioned on the left and bottom edges of the slide, framing the title text.

DATA STRUCTURES AND ALGORITHMS

Outline

- Arrays
- Linked List
- Operations
- Array vs. Linked List
 - *Pros and Cons*
- Types

Static vs Dynamic Data Structure

- An array is a structure of fixed-size data records such that each element can be efficiently located by its index or (equivalently) address.

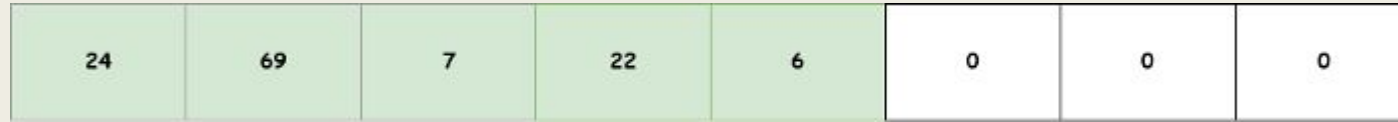
Advantages of contiguously-allocated arrays include:

- Constant-time access given the index.
- Arrays consist purely of data, so no space is wasted with links or other formatting information.

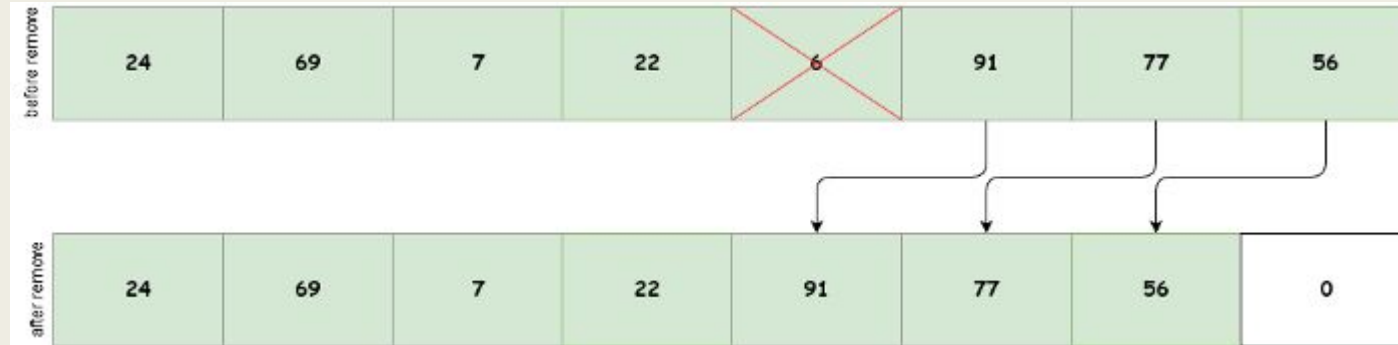
Array List

- The array list is basically a self-resizing array or, in other words, a dynamic array.
- This means that this data structure can grow as much as it needs — compared to the classical static array which cannot because it has a predefined maximum size.
- The need for this data structure is most obvious when you have the urge to **store items without knowing how much space** you'll need in advance.

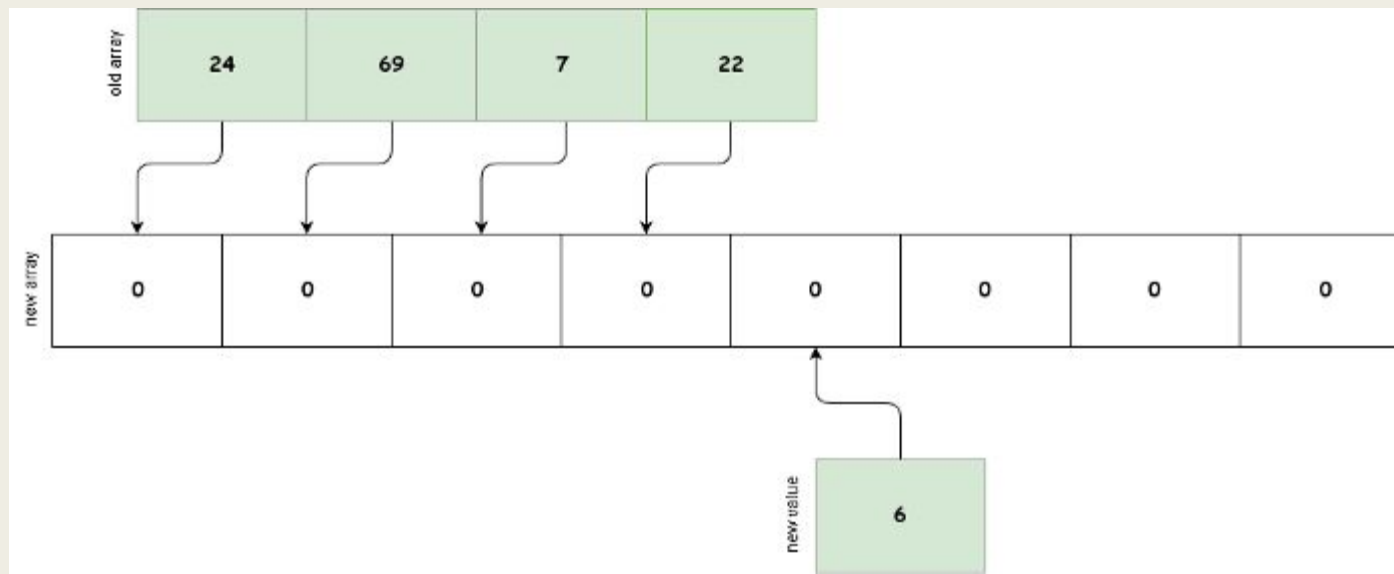
■ add(element)



■ remove(element)



■ resize(element)



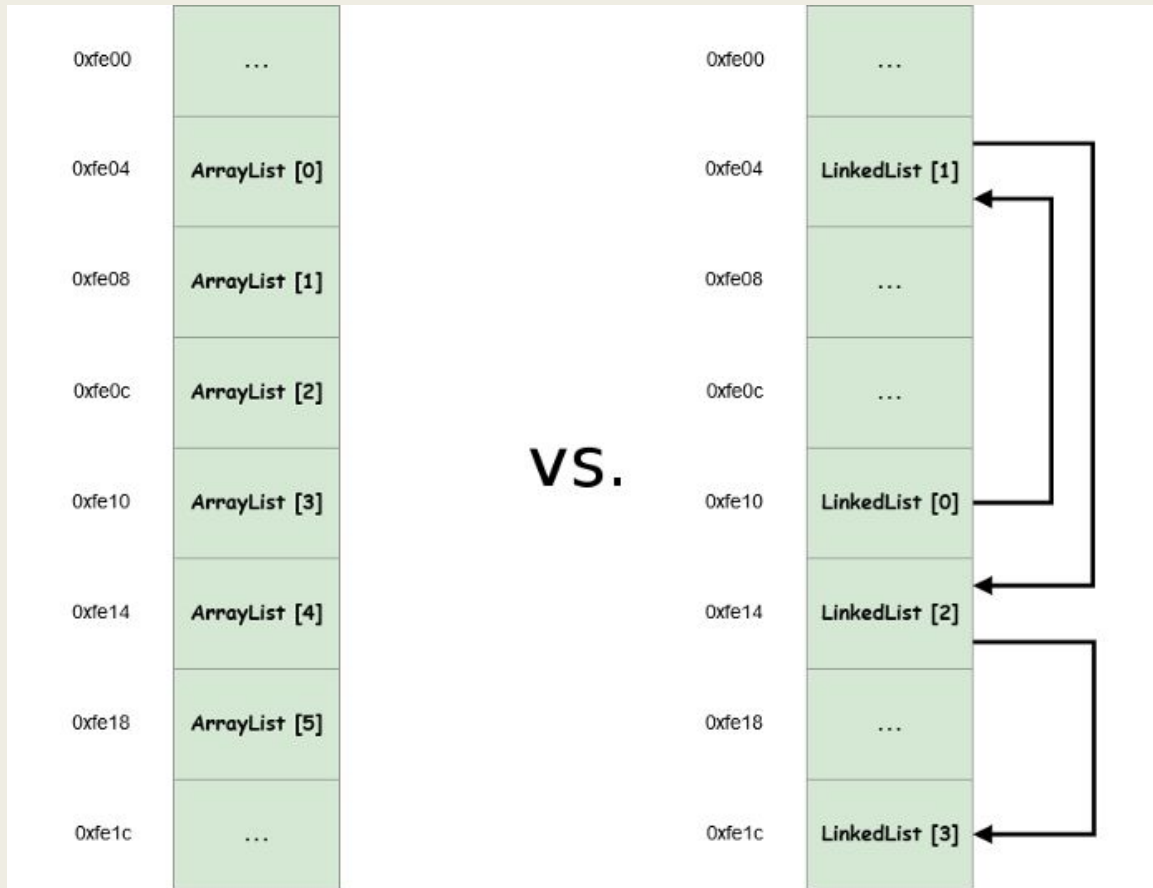
Array's Limitation

- Arrays

- *Simple,*
- *Fast*

but

- *Must specify size at construction time*
 - Construct an array with space for n
 - $n = \text{twice your estimate of largest collection}$
 - Tomorrow you'll need $n+1$
- *More flexible system?*

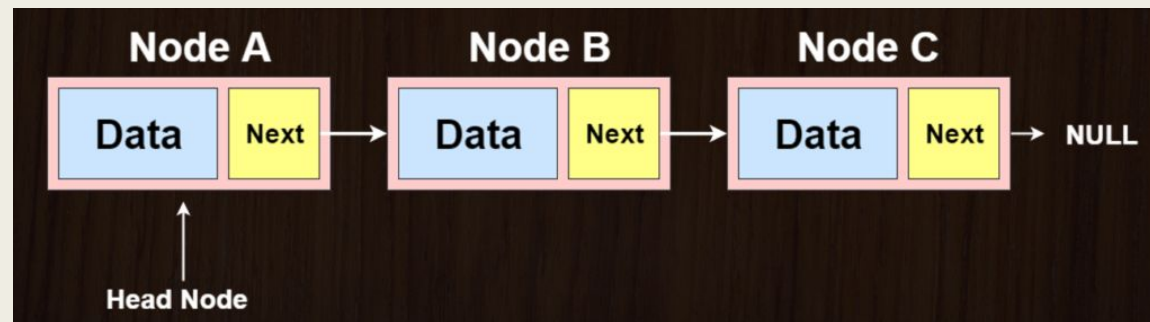


Therefore, there is need of a data structure that dynamically allocates space for each element as needed

□ **Linked List**

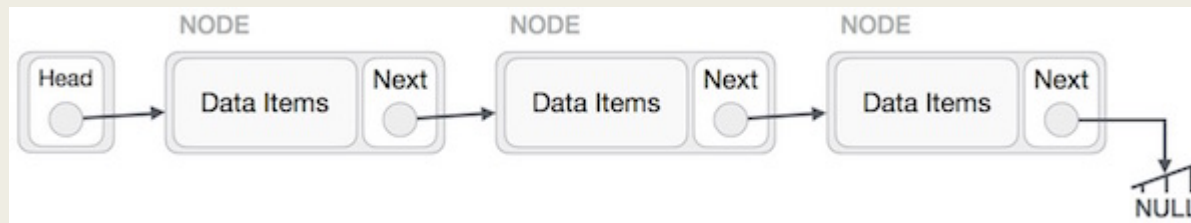
Linked List

- A linked list is a linear data structure, in which the elements are **not stored at contiguous memory locations**. The elements in a linked list are **linked** using **pointers**(entity that point to the next element)
- In simple words, a linked list consists of **nodes** where each node contains a data field and a **reference(link)** to the next node in the list.



Linked List Representation

Linked list can be visualized as a chain of nodes, where every node points to the next node.

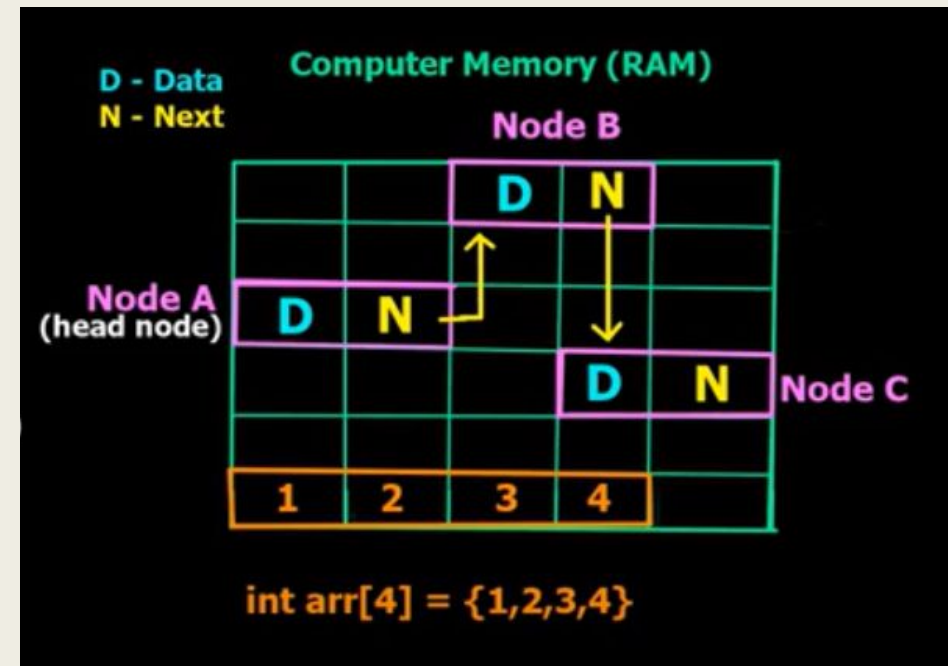


- As per the above illustration, following are the important points to be considered.
- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.

Working of Linked List

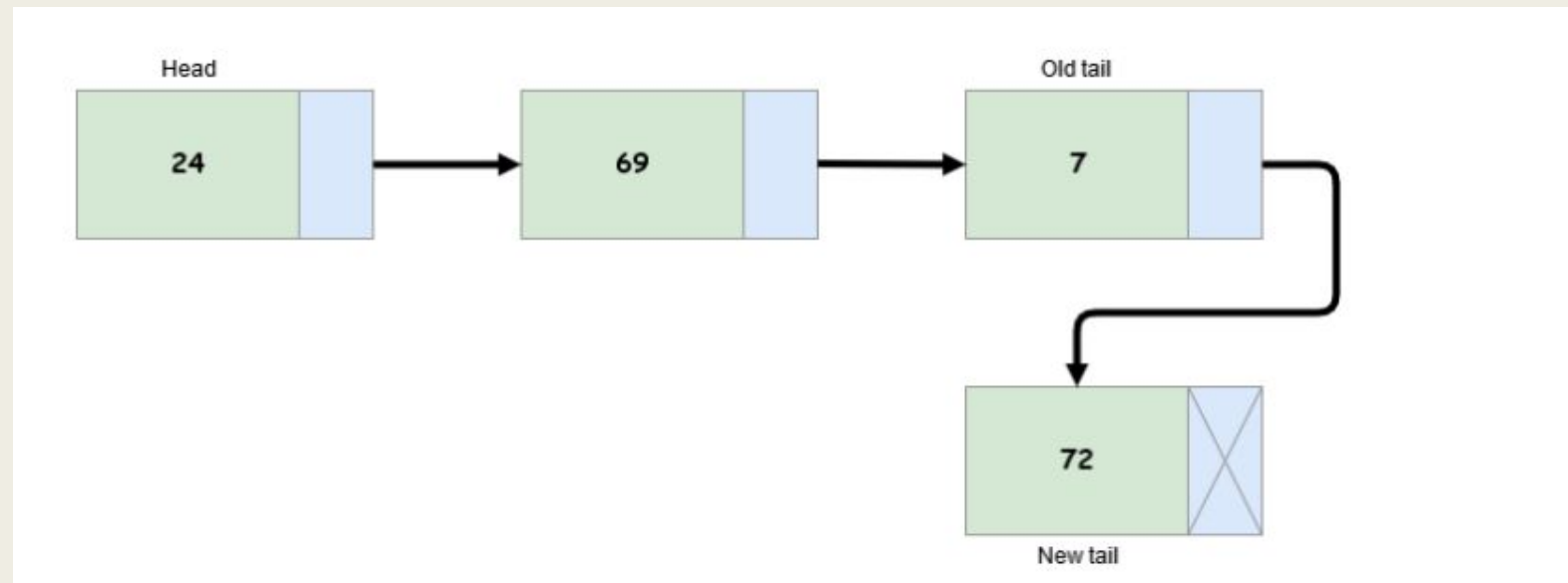
Linked list consists of nodes where each node contains a data field and a reference (link) to the next node in the link

- Node/Link/Element/Object – Each node in the linked list consists of two parts
 1. *Data*
 2. *Link to the next node*
- Next – This points to the next node in the linked list (since they are not stored in the contiguous memory locations)



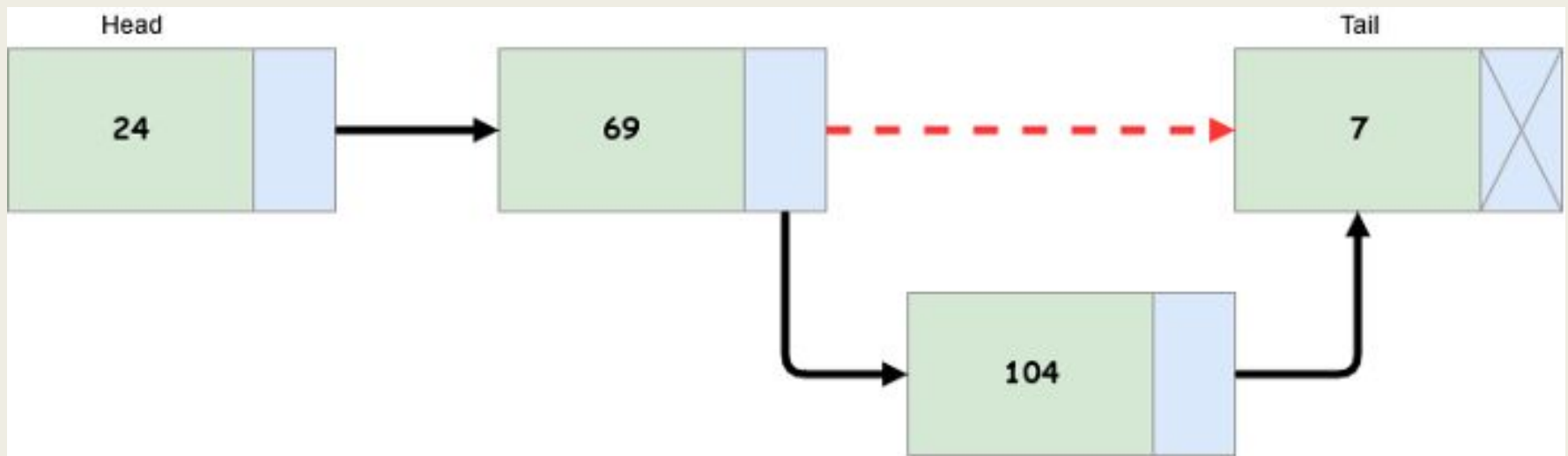
Operations on Linked List

■ Resize/Add



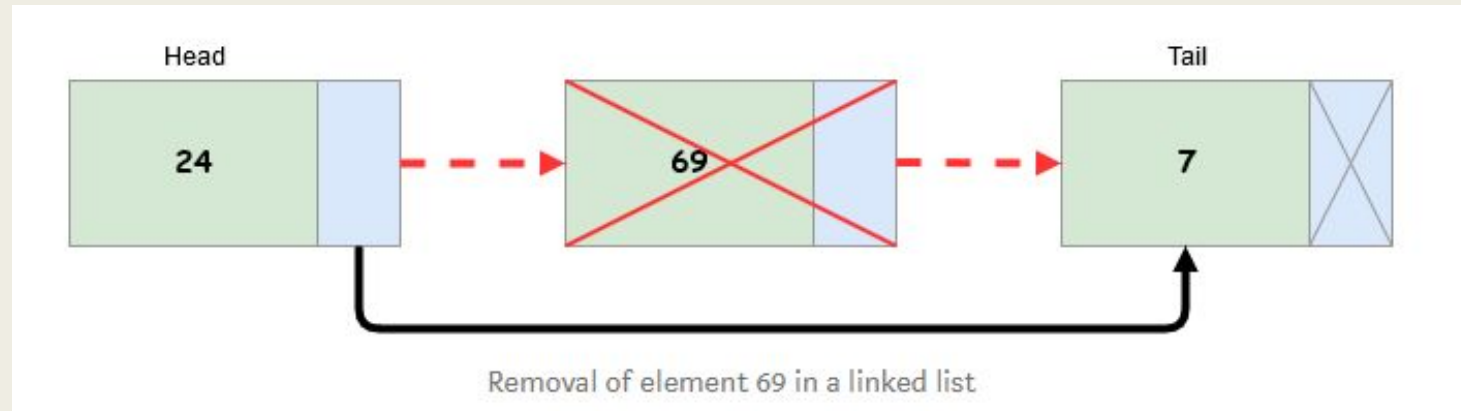
Operations on Linked List

■ Insert



Operations on Linked List

■ Remove



Operations of Linked List -

1. Traversing a linked list.



2. Append a new node (to the end) of a list



3. Prepend a new node (to start) of list



4. Inserting a new node to a specific position in the list



5. Deleting a node from the list



6. Updating a node in the list



Linked List vs Arrays

- Advantages of Linked List over Arrays

1. *Dynamic size*
2. *Ease of insertion/deletion*

- Disadvantages of Linked List over Arrays

1. *Random access is not allowed*
2. *Extra memory space for pointer*
3. *No cache friendly (array – contiguous)*

Types of Linked List

Types of Linked List -

D = Data P = Previous N = Next

1) Singly Linked List	2) Doubly Linked List	3) Circular Linked List
(head) 	(head) 	(head) 